Crop Disease Detector App - Documentation

Table of Contents

1. Project Overview
2. System Architecture
3. Technical Stack
4. Installation Guide
5. Project Structure
6. Core Features
7. User Guide
8. API Documentation
9. Database Schema
10. Security Implementation
11. Testing Guide
12. Deployment Guide
13. Troubleshooting
14. Future Enhancements

Project Overview

1.1 Introduction

Crop Disease Detector is a comprehensive mobile application built with Flutter that leverages artificial intelligence and machine learning to help farmers and agricultural enthusiasts detect crop diseases instantly through image analysis. The app provides detailed information about diseases, treatment recommendations, and allows users to share their findings with a community of farmers.

1.2 Purpose

- Enable instant crop disease detection using smartphone cameras
- Provide accurate treatment recommendations
- Build a community platform for farmers to share knowledge

- Store and track disease detection history
- Offer offline access to disease information

1.3 Target Audience

- Farmers and agricultural workers
- Agricultural students and researchers
- Gardening enthusiasts
- Agricultural extension officers
- Crop consultants

System Architecture

2.1 High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                      Client Layer (Flutter)                      │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐            │
│  │   UI     │ │  State   │ │  Local   │ │  Camera  │            │
│  │Components│ │Management│ │ Storage  │ │  Module  │            │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘            │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
```

```
┌──────────────────────────────────────────────────────────┐
│                    Service Layer (Dart)                  │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐    │
│  │   ML     │ │   Auth   │ │  Cloud   │ │ History  │    │
│  │ Service  │ │ Service  │ │ Service  │ │ Service  │    │
│  └──────────┘ └──────────┘ └──────────┘ └──────────┘    │
└──────────────────────────────────────────────────────────┘
                            │
                            ▼
┌──────────────────────────────────────────────────────────┐
│                    Backend Services                      │
│  ┌──────────────┐ ┌──────────────┐ ┌──────────────┐     │
│  │   Firebase   │ │  Firebase    │ │  Firebase    │     │
│  │  Auth        │ │  Firestore   │ │  Storage     │     │
│  └──────────────┘ └──────────────┘ └──────────────┘     │
│  ┌──────────────┐ ┌──────────────┐                      │
│  │  TensorFlow  │ │   ML Model   │                      │
│  │  Lite        │ │   Serving    │                      │
│  └──────────────┘ └──────────────┘                      │
└──────────────────────────────────────────────────────────┘
```

2.2 Architecture Patterns

MVVM (Model-View-ViewModel) Pattern:

- **Models**: Data classes representing business entities
- **Views**: UI screens and widgets
- **ViewModels**: Business logic and state management

Repository Pattern:

- Centralized data operations
- Abstraction between data sources and business logic

Service Layer Pattern:

- Encapsulated business services
- Reusable across the application

Technical Stack

3.1 Frontend (Flutter)

Technology	Version	Purpose
Flutter SDK	3.16.0+	Cross-platform framework
Dart	3.2.0+	Programming language
Provider	6.1.1	State management
GoRouter	14.2.7	Navigation
Hive	2.2.3	Local database
Image Picker	1.1.2	Camera/gallery access
TFLite Plus	1.0.3	ML model inference

3.2 Backend (Firebase)

Service	Purpose
Firebase Authentication	User management
Cloud Firestore	Database
Firebase Storage	Image storage
Firebase Analytics	Usage tracking
Firebase Crashlytics	Error reporting

3.3 Machine Learning

- **Model**: Custom-trained CNN model for plant disease detection
- **Framework**: TensorFlow Lite
- **Model Size**: ~15MB
- **Input Size**: 224x224 pixels
- **Supported Crops**: Tomato, Potato, Corn, Rice, Wheat
- **Diseases Detected**: 38 common crop diseases

3.4 Development Tools

Tool	Purpose

Android Studio	IDE
Visual Studio Code	Alternative IDE
Git	Version control
Postman	API testing
Firebase Console	Backend management
Figma	UI/UX design

Installation Guide

4.1 Prerequisites

bash

Required Software

1. Flutter SDK (>=3.16.0)

2. Android Studio / VS Code

3. Android SDK (for Android development)

4. Xcode (for iOS development, Mac only)

5. Git

6. Node.js (for Firebase CLI)

4.2 Environment Setup

Windows:

powershell

Add Flutter to PATH

setx PATH "%PATH%;C:\flutter\bin"

Verify installation

flutter doctor

Install additional dependencies

flutter pub global activate flutterfire_cli

macOS/Linux:

bash

Add Flutter to PATH

export PATH="$PATH:`pwd`/flutter/bin"

Verify installation

flutter doctor

Install Firebase CLI

curl -sL https://firebase.tools | bash

4.3 Project Setup

bash

cd crop_disease_detector

Install dependencies

flutter pub get

Configure Firebase

flutterfire configure

Run the app

flutter run

4.4 Environment Variables

Create .env file in project root:

env

FIREBASE_API_KEY=your_api_key

FIREBASE_APP_ID=your_app_id

FIREBASE_PROJECT_ID=your_project_id

FIREBASE_STORAGE_BUCKET=your_storage_bucket

ML_MODEL_PATH=assets/models/plant_disease_model.tflite

Project Structure

5.1 Directory Structure

```
crop_disease_detector/
├── android/              # Android native code
├── ios/                  # iOS native code
├── lib/
|   ├── core/
|   |   ├── constants/      # App constants
|   |   ├── themes/         # Theme configurations
|   |   ├── utils/          # Utility functions
|   |   └── routes/         # Navigation routes
|   ├── data/
|   |   ├── models/         # Data models
|   |   ├── repositories/   # Data repositories
|   |   └── services/       # Business services
|   ├── presentation/
|   |   ├── screens/        # UI screens
|   |   |   └── auth/       # Authentication screens
```

```
│   │   ├── viewmodels/     # State management
│   │   └── widgets/        # Reusable widgets
│   └── main.dart           # Entry point
├── assets/
│   ├── models/             # ML models
│   ├── images/             # Static images
│   └── icons/              # App icons
├── test/                   # Unit tests
├── docs/                   # Documentation
└── pubspec.yaml            # Dependencies
```

5.2 Key Files Description

File	Purpose
main.dart	Application entry point
auth_service.dart	Firebase authentication logic
ml_service.dart	TensorFlow Lite model inference
detection_service.dart	Cloud storage operations
history_service.dart	Local database operations
home_feed_screen.dart	Main social feed
camera_screen.dart	Camera capture interface
result_screen.dart	Disease analysis results

Core Features

6.1 User Authentication

Features:

- Email/Password registration

- Google Sign-In integration
- Email verification
- Password reset
- Profile management
- Session persistence

Implementation:

dart

```
// Sign up with email
Future<AppUser?> signUpWithEmail({
  required String email,
  required String password,
  required String username,
  String? displayName,
  String? phoneNumber,
  DateTime? dateOfBirth,
}) async {
  // Implementation details
}
```

6.2 Disease Detection

Process Flow:

1. Capture/select image
2. Image preprocessing (resize, normalize)
3. ML model inference
4. Confidence scoring
5. Disease identification
6. Treatment recommendations

Supported Diseases:

Crop	Diseases
Tomato	Early Blight, Late Blight, Leaf Mold, Septoria Leaf Spot
Potato	Early Blight, Late Blight, Healthy
Corn	Gray Leaf Spot, Common Rust, Northern Leaf Blight
Rice	Brown Spot, Leaf Blast, Bacterial Blight
Wheat	Stripe Rust, Leaf Rust, Stem Rust, Septoria

6.3 Social Features

- **Public Feed**: View detections from other users
- **Like System**: Interact with community posts
- **Comments**: Discuss detections and treatments
- **Sharing**: Share results via social media
- **Following**: Follow other farmers and experts

6.4 History & Statistics

- **Local Storage**: Offline access to past detections
- **Cloud Sync**: Cross-device history access
- **Statistics**: Disease frequency analysis
- **Search**: Filter and search past detections
- **Export**: Share history as CSV or screenshot

User Guide

7.1 Getting Started

Step 1: Create Account

1. Launch the app
2. Tap "Sign Up"
3. Enter required information
4. Verify email address

5. Complete profile setup

Step 2: First Scan

1. Tap "Scan Now" button
2. Allow camera permissions
3. Take photo of crop leaf
4. Wait for analysis (2-3 seconds)
5. View results and recommendations

Step 3: Explore Feed

1. Navigate to Home tab
2. Browse community detections
3. Like and comment on posts
4. Follow interesting users

7.2 Screen Guide

Home Feed Screen

#add home screen feed picture

Camera Screen

#add camera scan picture

Result Screen

#Add results screen picture

7.3 Keyboard Shortcuts (Development)

Shortcut	Action
r	Hot reload
R	Hot restart
p	Show performance overlay
o	Open on device

h	Help menu
q	Quit app

API Documentation

8.1 Firebase Authentication

Endpoints:

dart

```
// Sign up
POST /v1/accounts:signUp
Body: {
  "email": "user@example.com",
  "password": "password123",
  "returnSecureToken": true
}

// Sign in
POST /v1/accounts:signInWithPassword
Body: {
  "email": "user@example.com",
  "password": "password123",
  "returnSecureToken": true
}
```

8.2 ML Service API

dart

```
class MLService {
  // Load model
```

```
  Future<void> loadModel();

  // Predict disease
  Future<List<PredictionModel>> predictImage(Uint8List imageBytes);

  // Dispose model
  Future<void> disposeModel();
}

class PredictionModel {
  final String label;
  final double confidence;
}
```

8.3 Firestore Collections

Users Collection:

javascript

```
users/{userId} {
  uid: string,
  email: string,
  username: string,
  displayName: string,
  photoURL: string,
  createdAt: timestamp,
  lastLoginAt: timestamp,
  followers: array,
  following: array,
```

```
  totalDetections: number
}
```

Detections Collection:

javascript

```
detections/{detectionId} {
  userId: string,
  username: string,
  diseaseName: string,
  confidence: number,
  imageURL: string,
  timestamp: timestamp,
  likes: number,
  comments: number,
  isPublic: boolean,
  tags: array
}
```

Database Schema

9.1 Local Database (Hive)

Box: detection_history

dart

```
@HiveType(typeId: 0)
class DetectionHistory {
  @HiveField(0)
  String id;
```

```
  @HiveField(1)
  String imagePath;

  @HiveField(2)
  String diseaseName;

  @HiveField(3)
  double confidence;

  @HiveField(4)
  DateTime timestamp;
}
```

9.2 Cloud Database (Firestore)

Collection: users

```
users/{userId}
├── email: string
├── username: string
├── displayName: string (optional)
├── phoneNumber: string (optional)
├── photoURL: string (optional)
├── isEmailVerified: boolean
├── createdAt: timestamp
├── lastLoginAt: timestamp
├── followers: array<string>
├── following: array<string>
├── totalDetections: number
```

└── settings: map

Collection: detections

detections/{detectionId}

├── userId: string

├── username: string

├── userPhotoURL: string (optional)

├── diseaseName: string

├── confidence: number

├── imageURL: string

├── localImagePath: string (optional)

├── diseaseDetails: map (optional)

├── timestamp: timestamp

├── notes: string (optional)

├── location: map (optional)

├── likes: number

├── likedBy: array<string>

├── comments: number

├── isPublic: boolean

└── tags: array<string>

Security Implementation

10.1 Authentication Security

- Email verification required
- Password strength requirements (min 6 chars)
- Rate limiting on login attempts
- Session management with automatic timeout

- Secure token storage

10.2 Data Security

- End-to-end encryption for sensitive data
- Firestore security rules
- Firebase Storage security rules
- Local database encryption with Hive

10.3 Firestore Security Rules

javascript

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Users collection
    match /users/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null && request.auth.uid == userId;
    }

    // Detections collection
    match /detections/{detection} {
      allow read: if resource.data.isPublic == true ||
                  request.auth.uid == resource.data.userId;
      allow write: if request.auth.uid == resource.data.userId;
      allow create: if request.auth != null;
    }
  }
}
```

10.4 Storage Security Rules

javascript

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    match /detections/{userId}/{fileName} {
      allow read: if true;
      allow write: if request.auth.uid == userId;
    }

    match /profiles/{userId}/{fileName} {
      allow read: if true;
      allow write: if request.auth.uid == userId;
    }
  }
}
```

Testing Guide

11.1 Unit Testing

dart

```
// test/models/detection_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:crop_disease_detector/data/models/detection_model.dart';

void main() {
  group('Detection Model Tests', () {
```

```
  test('Detection creation works correctly', () {
    final detection = Detection(
      id: '123',
      userId: 'user123',
      username: 'testuser',
      diseaseName: 'Early Blight',
      confidence: 0.95,
      imageURL: 'https://example.com/image.jpg',
      timestamp: DateTime.now(),
    );

    expect(detection.id, '123');
    expect(detection.diseaseName, 'Early Blight');
    expect(detection.confidence, 0.95);
  });
});
}
```

11.2 Widget Testing

dart

```
// test/widgets/result_screen_test.dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:provider/provider.dart';
import 'package:crop_disease_detector/presentation/screens/result_screen.dart';

void main() {
```

```
  testWidgets('Result screen displays correctly', (tester) async {
    await tester.pumpWidget(
      MaterialApp(
        home: ResultScreen(),
      ),
    );

    expect(find.text('Analysis Result'), findsOneWidget);
  });
}
```

11.3 Integration Testing

dart

```
// test_driver/app_test.dart
import 'package:flutter_driver/flutter_driver.dart';
import 'package:test/test.dart';

void main() {
  group('Crop Disease Detector App', () {
    FlutterDriver? driver;

    setUpAll(() async {
      driver = await FlutterDriver.connect();
    });

    tearDownAll(() async {
      driver?.close();
```

```
  });

  test('Home screen loads', () async {
    expect(await driver.getText(find.byValueKey('home_title')), 'Crop Disease Detector');
  });
});
}
```

11.4 Test Coverage

bash

```
# Run all tests
flutter test

# Run with coverage
flutter test --coverage

# Generate coverage report
genhtml coverage/lcov.info -o coverage/html
```

Deployment Guide

12.1 Android Deployment

Step 1: Generate Keystore

bash

```
keytool -genkey -v -keystore upload-keystore.jks -keyalg RSA -keysize 2048 -validity 10000 -alias upload
```

Step 2: Configure Signing

gradle

```
// android/app/build.gradle
android {
    signingConfigs {
        release {
            keyAlias keystoreProperties['keyAlias']
            keyPassword keystoreProperties['keyPassword']
            storeFile keystoreProperties['storeFile'] ? file(keystoreProperties['storeFile']) : null
            storePassword keystoreProperties['storePassword']
        }
    }

    buildTypes {
        release {
            signingConfig signingConfigs.release
        }
    }
}
```

Step 3: Build APK

bash

```
flutter build apk --release
# Output: build/app/outputs/flutter-apk/app-release.apk
```

Step 4: Build App Bundle

bash

```
flutter build appbundle --release
# Output: build/app/outputs/bundle/release/app-release.aab
```

12.2 iOS Deployment

Step 1: Configure Xcode

bash

open ios/Runner.xcworkspace

Set bundle identifier

Configure signing team

Step 2: Build IPA

bash

flutter build ios --release

Archive in Xcode

Step 3: Submit to App Store

bash

Use Xcode or Transporter app

12.3 Environment Configurations

Development:

yaml

flutter_launcher_icons.yaml

flutter_launcher_icons:

  image_path: "assets/icons/dev_icon.png"

  android: **true**

  ios: **true**

Production:

yaml

flutter_launcher_icons.yaml

flutter_launcher_icons:

  image_path: "assets/icons/app_icon.png"

  android: **true**

ios: **true**

Troubleshooting

13.1 Common Issues & Solutions

Issue	Solution
App crashes on launch	Run flutter clean && flutter pub get
Camera permission denied	Check Android/iOS manifest files
ML model not loading	Verify model path in assets
Firebase connection failed	Check google-services.json placement
Build fails on iOS	Update Podfile and run pod install

13.2 Debugging Commands

bash

Verbose build

flutter run -v

Check dependencies

flutter pub deps

Analyze code

flutter analyze

Check for outdated packages

flutter pub outdated

Upgrade packages

flutter pub upgrade --major-versions

13.3 Performance Optimization

dart

```
// Use const constructors
const Widget myWidget = const MyWidget();

// Implement lazy loading
ListView.builder(
  itemCount: items.length,
  itemBuilder: (context, index) => MyListItem(items[index]),
)

// Cache images
CachedNetworkImage(
  imageUrl: 'https://example.com/image.jpg',
  placeholder: (context, url) => CircularProgressIndicator(),
)
```

Future Enhancements

14.1 Planned Features

Phase 7 (Q2 2026)

- Offline mode with local ML inference
- Multi-language support (Spanish, French, Hindi)
- Weather integration for disease prediction
- Push notifications for disease alerts

Phase 8 (Q3 2026)

- AR visualization of disease spread

- Integration with agricultural weather APIs
- Marketplace for organic treatments
- Expert consultation booking system

Phase 9 (Q4 2026)

- IoT device integration
- Drone imagery analysis
- Blockchain for crop certification
- AI-powered treatment recommendations

14.2 Scalability Considerations

- Implement CDN for image delivery
- Add database indexing for faster queries
- Implement caching layer with Redis
- Use load balancing for API endpoints
- Implement pagination for feeds

14.3 Monetization Strategies

- Premium features for advanced analytics
- Subscription model for unlimited scans
- Sponsored content from agricultural companies
- Marketplace commissions
- API access for enterprise clients

Support & Contact

Technical Support

- **Email**: support@cropdiseaseapp.com

Community

- **Discord**: https://discord.gg/cropdisease

- **Twitter**: @CropDiseaseApp
- **Facebook**: /CropDiseaseDetector

License

This project is licensed under the MIT License - see the LICENSE file for details.

Version History

Version	Date	Changes
1.0.0	Feb 2026	Initial release with basic detection
1.1.0	Mar 2026	Added social features
1.2.0	Apr 2026	Multi-crop support
1.3.0	May 2026	Offline mode
2.0.0	Jun 2026	Community features and cloud sync

Contributors

- **Lead Developer**: [Your Name]
- **ML Engineer**: [Name]
- **UI/UX Designer**: [Name]
- **Agricultural Expert**: [Name]

This documentation was last updated on February 23, 2026

End of Documentation